YAN ALMEIDA

Software & Systems Engineering · Backend (.NET / C# / C / Python)
Vila Nova de Gaia, Porto, Portugal | yancsalmeida@gmail.com | linkedin.com/in/yancsa | github.com/yancsalmeida | yancsalmeida.github.io

PROFILE

Software Engineering student (**CTeSP TPSI at ISPGAYA**, Porto) with a quantitative background in **Chemical Engineering** (**BSc from Budapest University of Technology and Economics — BME**). Focused on backend development in **C# (.NET 8)**, systems programming in **C11**, and algorithmic modeling in **Python**. Translates mathematical modeling, differential equations, and process control principles into clean software architecture, high-throughput data ingestion pipelines, and low-level memory efficiency.

TECHNICAL STACK

- **Languages:** C# (.NET 8), C (C11, pointers, manual memory management), Python 3 (Asyncio), SQL, Bash
- **Backend & Data:** ASP.NET Core Web API, Entity Framework Core, Dapper, Npgsql (PostgreSQL Binary `COPY`), `System.Threading.Channels`, SignalR WebSockets, FastAPI, Pydantic v2, PostgreSQL 16, SQLite
- **Systems & Architecture:** Clean Architecture, SOLID, Native C-ABI Interop (C# P/Invoke, Python `ctypes`), Directed Acyclic Graphs (Kahn's Topological Sort), Numerical Solvers (Runge-Kutta 4, Newton-Raphson, TDMA)
- **Infrastructure & Testing:** Linux (Ubuntu), Docker, Docker Compose, Git, GitHub Actions (CI), xUnit, Pytest, Serilog

SELECTED SOFTWARE PROJECTS

IndustrialTelemetry.Engine
C# 12 · .NET 8 · PostgreSQL 16 · Clean Architecture · Docker

Asynchronous Telemetry Ingestion Pipeline & Batch Execution State Machine

- Built a decoupled producer-consumer ingestion pipeline using bounded in-memory channels (`System.Threading.Channels`) and a hosted background worker (`BackgroundService`) with micro-batch flushing.
- Implemented direct PostgreSQL Binary `COPY` persistence (`NpgsqlBinaryImporter`) with composite indexing on `batch_id` and `timestamp_utc` to minimize memory allocations and write latency under high event loads.
- Modeled strict batch lifecycle transitions (`Scheduled` → `InProgress` → `Completed/Aborted`) with compliance auditing and real-time threshold alarm rules.

ChemFlow.NET
C# 12 · .NET 8 · Clean Architecture · SignalR · C11 P/Invoke · SQLite

Modular Process & Thermodynamic Simulation Engine

- Implemented an analytical Peng-Robinson cubic equation of state (Cardano solver) and a safeguarded Newton-Raphson Rachford-Rice vapor-liquid equilibrium flash solver.
- Developed a rigorous multicomponent staged Distillation Column (MESH solver) using the Thomas Algorithm (TDMA) and Holland's θ-method, along with a DAG flowsheet executor using Kahn's topological sort and Wegstein recycle acceleration.
- Integrated native C11 ODE integration via P/Invoke and real-time simulation telemetry streaming via ASP.NET Core SignalR.

ProcessTwin.Py
Python 3.12 · FastAPI · Asyncio · C11 FFI (`ctypes`) · WebSockets

Real-Time Process Digital Twin & Supervisory Telemetry Service

- Connected Python asynchronous simulation loops to a compiled C11 shared library (`libreactorcore.so`) via `ctypes` FFI, achieving a 9.4x batch execution speedup with IEEE-754 numerical parity.
- Emulated 4–20 mA sensor drift/noise, implemented automated safety interlocks (IEC 61511 SIS logic), and streamed 5 Hz state updates over WebSockets to an interactive supervisory interface.

ReactorCore.C | cs-fundamentals
C11 · GCC · C# Concurrency · Linux `x86_64`

Native Numerical ODE Solver & Low-Level Systems Laboratories

- Authored a zero-dependency C11 dynamic reactor solver using classical 4th-Order Runge-Kutta (RK4) via function pointers and contiguous workspace pre-allocation (zero heap allocation inside integration loops), exported as a shared C-ABI library.
- Developed systems laboratories exploring Linux `x86_64` virtual memory layouts, pointer arithmetic, and thread synchronization primitives in C# and C11.

EDUCATION

- **CTeSP in Information Systems Technologies and Programming (TPSI)** 2026 – 2028
  ISPGAYA (Instituto Superior Politécnico Gaya) — Vila Nova de Gaia, Porto, Portugal
  Core track: Algorithms & Data Structures (C, C#), Computer Architecture, Relational Databases (SQL/PostgreSQL), Networks, Web APIs.
- **Bachelor of Science (BSc) in Chemical Engineering** 2022 – 2026
  Budapest University of Technology and Economics (BME / Műegyetem) — Budapest, Hungary
  Conducted entirely in English. Completed 15 MSc-level graduate courses, including *Differential Equations (Mathematics M1c)*, *Electronics & Measurement Technology*, and *Chemical Production Control*.
- **Technical Diploma in Chemistry** 2015 – 2018
  IFRJ (Instituto Federal do Rio de Janeiro) — Rio de Janeiro, Brazil

LANGUAGES

Portuguese: Native | **Spanish:** Native / Bilingual | **English:** Full Professional Proficiency (4-year BSc in English)